Curso: Compiladores
Tema 2. Analizador léxico.

Asignación 1: Implementación del analizador léxico de su proyecto final en Windows

Prof. David Martínez Torres
Instituto de Computación
Universidad Tecnológica de la Mixteca

Introducción

En la presente asignación se desarrollará el analizador léxico, que es la primera fase de un compilador, en este caso, será la primera parte del compilador que se desarrollará en el sistema operático Windows. Para esto, tendrá que diseñar un autómata finito determinista (AFD), que reconocerá todas las cadenas válidas (componentes léxicos) definidas para una parte de un lenguaje fuente de programación de alto nivel que cada equipo elegirá.

Es recomendable realizar la implementación en un IDE (Entorno de Desarrollo Integrado) como Eclipse, Netbeans, BorlandC++, Python, etc. Porque disminuirá el trabajo de desarrollo y presentará una interfaz amigable.

Requisitos:

1. **Seleccionar el lenguaje fuente de programación (de alto nivel), al cual se le aplicará el analizador léxico.** De esta forma podrá definir las expresiones regulares que reconozcan las cadenas de símbolos de dicho lenguaje. Es importante identificar si el lenguaje de programación es estructurado u orientado a objetos.
2. Diseñe el AFD y obtenga una tabla de estado finales con sus correspondientes tokens.
3. Escriba ejemplos de programas para que identifique los componentes léxicos. En este documento se tomó como base **componentes léxicos corresponden del lenguaje C.** También, se ha definido que los programas a compilar solo cuenten con la función principal y otras funciones, declaración de variables simples, arreglos, apuntadores simples y doble apuntador, tipos primitivos, instrucciones principales de entrada/salida, y todas las estructuras de control con llaves y sin llaves, dependiendo de la cantidad de instrucciones en el

cuerpo de la estructura de control. Opcional considerar librerías o directivas de preprocesador. **Se adjunta ejemplo del tipo programa**.

4. Definir el alfabeto del lenguaje fuente.
5. Ejemplo de expresiones regulares del lenguaje de programación:

   a. **Identificador**

      id=letra(letra|digito)*
      donde:
      letra= a-zA-z
      digito= 0-9
      Ejemplos: suma, cont1, fibo, i, etc.

   b. **Palabras reservadas** o funciones del lenguaje de programación no considerarlas como identificadores, considerarlas propiamente como palabras reservadas. Todas las palabras reservadas puede guardarlas en un archivo o en una estructura de datos. Ejemplo: if, else, switch, while, do, for, main, printf, scanf, pow, int, float, etc.

   c. Llave que abre

      { = {

   d. y llave que cierra

      } = }

   e. **Número entero**

      nint=digito+
      Ejemplo: 10,1,30,200, etc.

   f. Número reale

      nfloat= digito*.digito+
      Ejemplo: 2.3, 100.32, etc

   g. literal cadena

      literalCad="(letra|digito|caracterEspecial)*"
      Ejemplo: “teclea un entero:”, “bienvenidos”, etc

   h. literal caracter

      literalCar=’(letra|digito|caracterEspecial)’
      Ejemplo: ‘a’ , ‘b’, ‘z’, etc.

   i. Coma

      , = ,

   j. Punto y coma

      ; = ;

k. Paréntesis que abre

(= (

l. Paréntesis que cierra

) =)

m. Corche que abre

[= [

n. Corchete que cierra

] =]

o. Operadores aritméticos

+ = +

- = -

* = *

/ = /

p. Operador de incremento

++ = ++

q. Operador de decremento

-- = --

r. Operador de asignación

= = =

s. Operadores abreviados

+= = +=

-= = -=

***= = *=**

/= = /=

t. Operadores relacionales

< = <

<= = <=

> = >

>= = >=

== = ==

!= = !=

u. Operador de estructuras

. = .

-> = ->

v. Operador de dirección

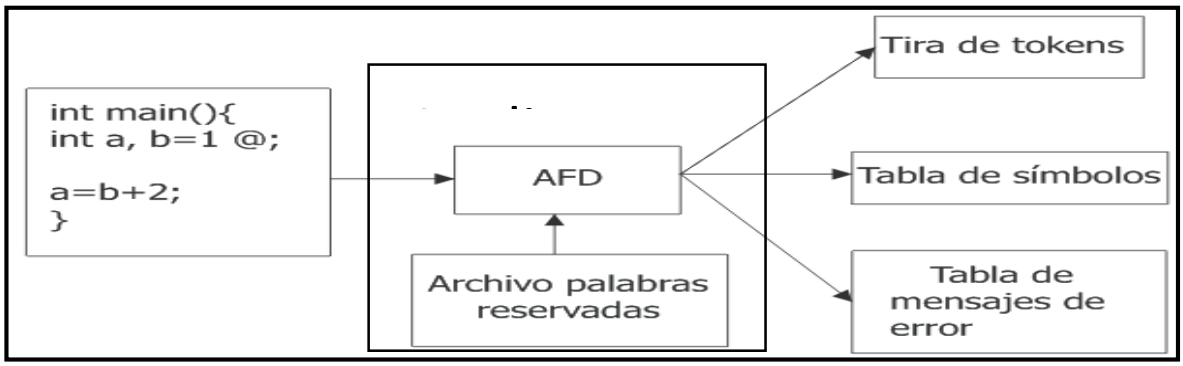
& = &

w. Operadores lógicos

&& = &&
||= ||
! = !

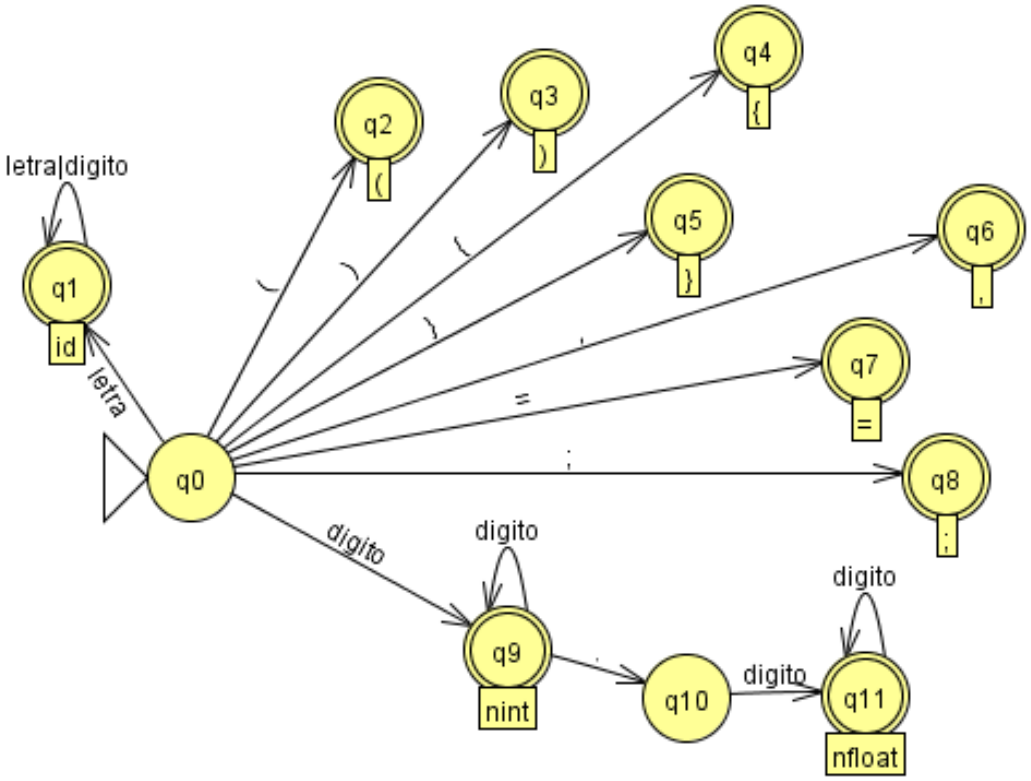
6. El analizador léxico debe descartar: Comentarios de línea, comentarios de bloque, espacios en blanco, tabuladores, etc.
7. Construir el analizador léxico basado en el AFD, para que **genere las salidas** del análisis léxico: **tira de tokens tabla de símbolos y tabla de errores**.

Bosquejo de solución



Ejemplo de algunas expresiones regulares iniciales y su respectivo AFD, que reconocen tokens del analizador léxico:

letra(letra|digito)* | (|) | { | } | , | = | ; | digito+(.digito+)?



El AFD puede obtenerse idealmente de los algoritmos de Thompson y de Construcción de conjuntos, sin embargo, también puede optar por construirlo como este ejemplo.

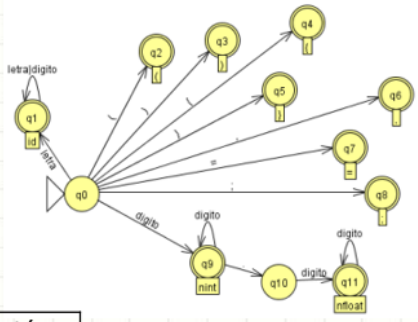
A continuación se muestran las salidas del analizador léxico del siguiente programa fuente en el lenguaje C.

Programa fuente

```
int main(){
int a, b=1 @;

a=b+2;
}
```

AFD



1. Tira de tokens

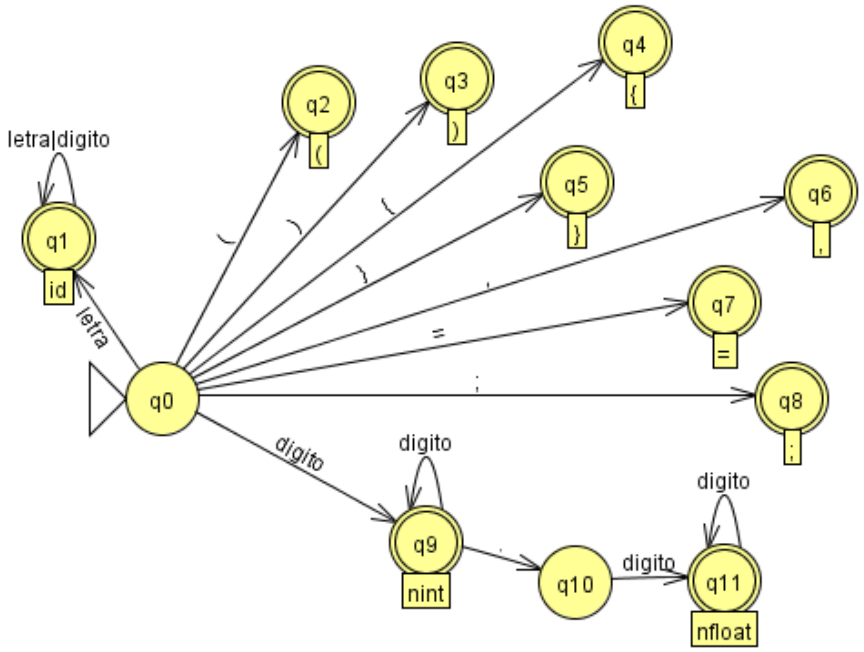
# línea	Lexema	Token
1	int	int
1	main	main
1	(	(
1	)	)
1	{	{
2	int	int
2	a	id.”a”
2	,	,
2	b	id.”b”
2	=	=
2	1	nint.1
2	;	;
4	a	id.”a”
4	=	=
4	b	id.”a”
4	+	+
4	2	nint.2
4	;	;
5	}	}

2. Tabla de símbolos

id	valor	tipo	función
a b			

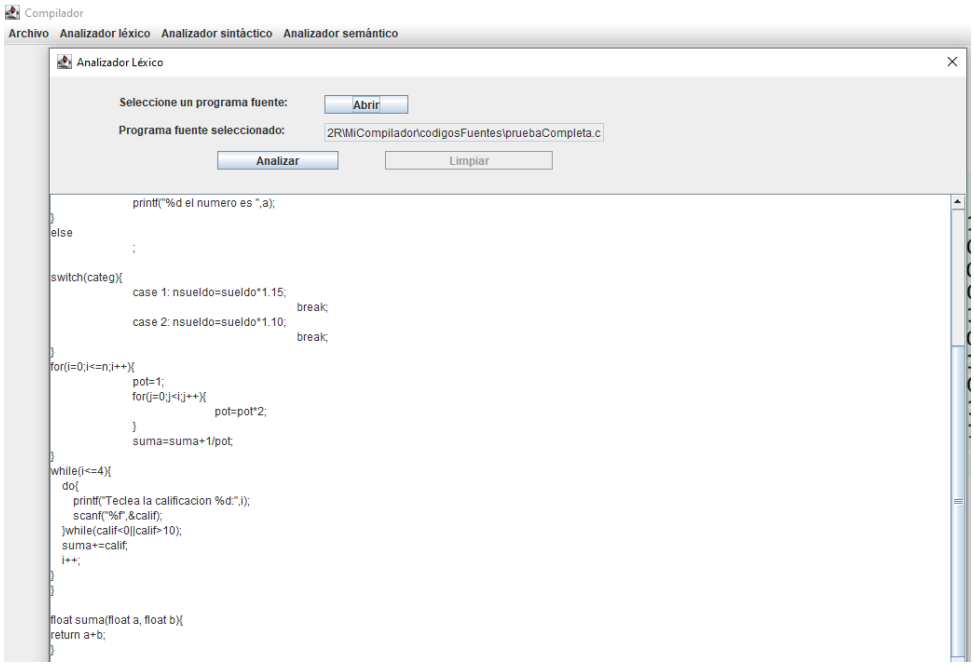
3. Tabla de errores

# línea	Descripción
	En función ‘main’
2	Error: @ símbolo no definido.

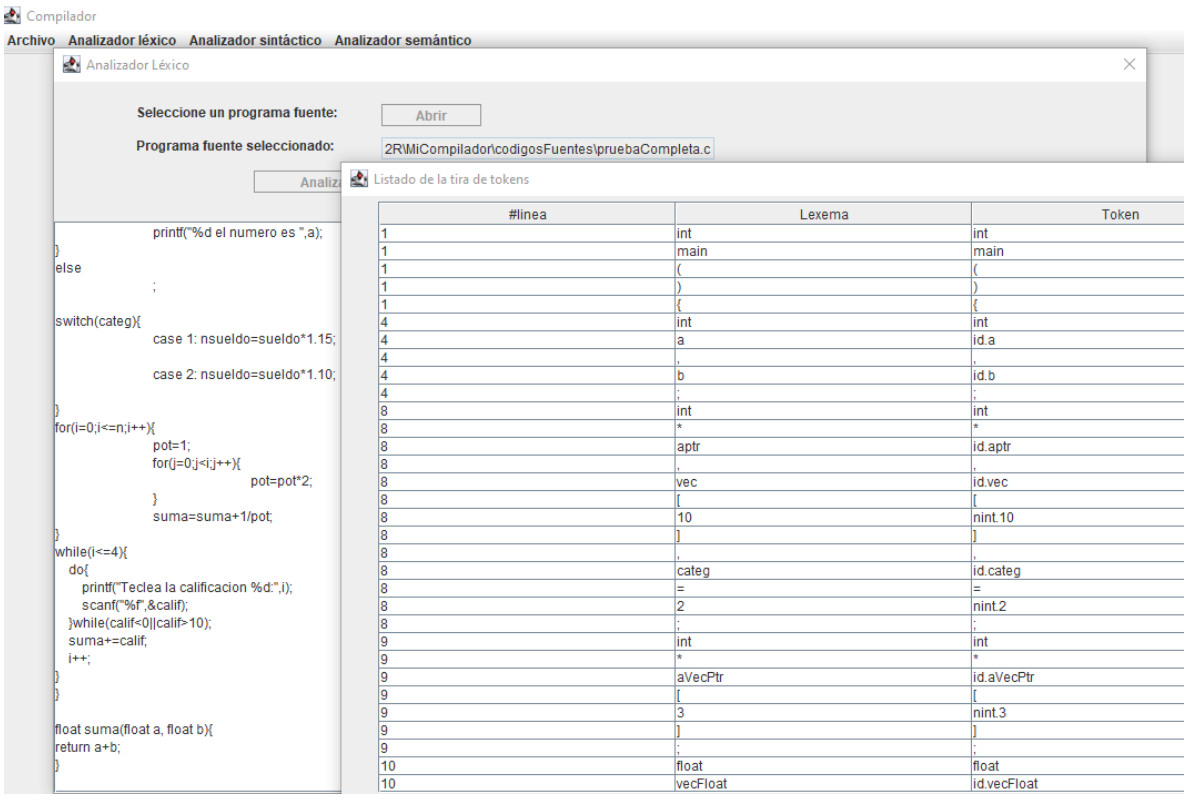


Ejemplo de posibles interfaces del análisis léxico.

1. Abrir el programa fuente a realizar análisis léxico

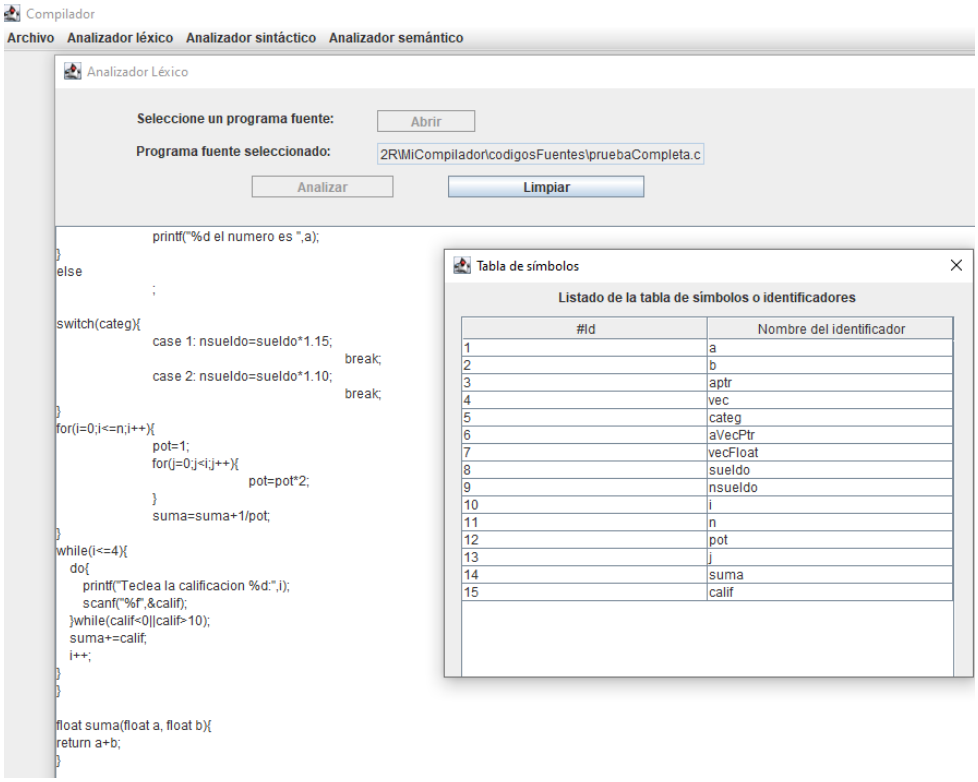


2. Salida de la tira de tokens. Recuerde que para el token id, nint, nfloat, literalCad y literalCar, imprima el lexema como valor léxico del token, ver ejemplos en la siguiente lista.

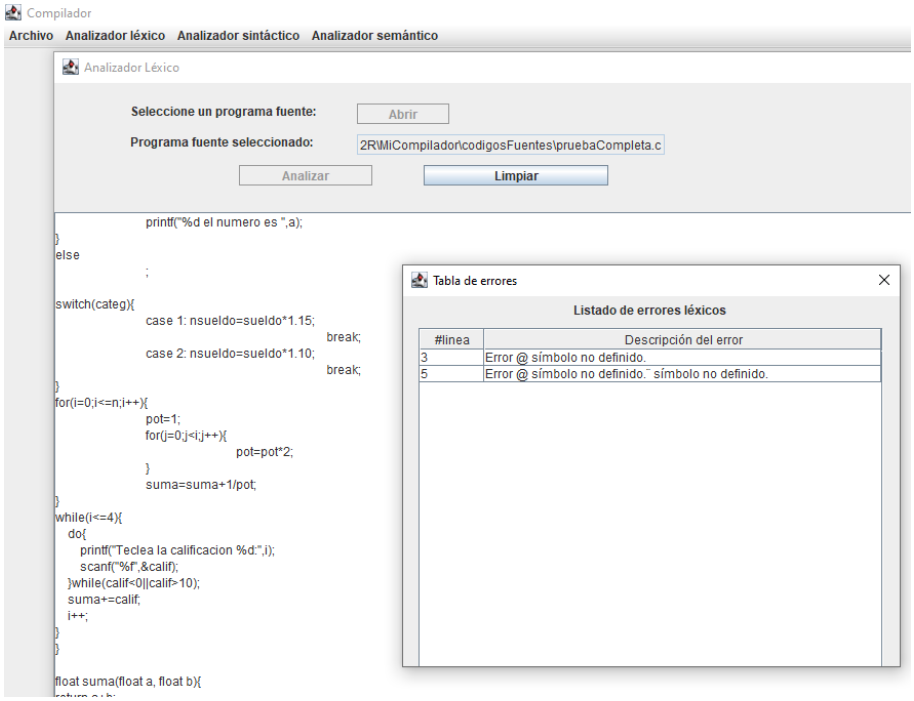


#linea	Lexema	Token
1	int	int
1	main	main
1	(	(
1	)	)
1	{	{
4	int	int
4	a	id.a
4	,	,
4	b	id.b
4	;	;
8	int	int
8	*	*
8	aptr	id.aptr
8	,	,
8	vec	id.vec
8	[	[
8	10	nint.10
8	]	]
8	,	,
8	categ	id.categ
8	=	=
8	2	nint.2
8	;	;
9	int	int
9	*	*
9	aVecPtr	id.aVecPtr
9	[	[
9	3	nint.3
9	]	]
9	;	;
10	float	float
10	vecFloat	id.vecFloat

3. Salida de tabla de símbolos, aquí mostrar por cada función de su programa



4. Salida de tabla de errores.



Fin de la Asignación